

Sapient

[Print](#) 

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	Staking Protocol
Timeline	2025-01-23 through 2025-01-23
Language	Solidity
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review
Specification	None
Source Code	https://github.com/Sapient-io/sapient-contracts  #c05fcbb 
Auditors	- Julio Aguilar Auditing Engineer - Mostafa Yassin Auditing Engineer - Tim Sigl Auditing Engineer - Paul Clemson Auditing Engineer

Documentation quality	Medium 
Test quality	Low 

Total Findings	17 Fixed: 12 Acknowledged: 5
High severity findings ⓘ	2 Fixed: 2
Medium severity findings ⓘ	6 Fixed: 4 Acknowledged: 2
Low severity findings ⓘ	6 Fixed: 3 Acknowledged: 3
Undetermined severity findings ⓘ	0
Informational findings ⓘ	3 Fixed: 3

Summary of Findings

Fix Review Update The client implemented fixes for all critical issues of the protocol.

Initial Audit Sapien is a staking protocol that allows users to stake and earn rewards based on their lockup period. Rewards are distributed by the admin and not automatically accrued when user interact with the protocol. Users can unstake by initiating an unstake process, and then wait for a cooldown before unstaking.

The audit identified multiple critical vulnerabilities in the protocol, and it is recommended that they be fixed by applying the recommendations and redesigning the vulnerable components to comply with well-known design patterns for reward protocols.

ID	SAP-1
DESCRIPTION	Missing Balance Deduction in Unstaking Functions Allows Contract Drainage and Lock Period Bypass
SEVERITY	High ⓘ
STATUS	Fixed

ID	SAP-2
DESCRIPTION	Signature Replay Attack Possible Between Stake, Unstake and Reward Functions Enabling Unauthorized Token Claims
SEVERITY	High ⓘ
STATUS	Fixed

ID	SAP-3
DESCRIPTION	Bloom Filter Implementation Can Lead to False Positives Preventing Valid Reward Claims
SEVERITY	Medium ⓘ
STATUS	Fixed

ID	SAP-4
DESCRIPTION	Emergency Pause Functionality Not Implemented
SEVERITY	Medium ⓘ
STATUS	Fixed

ID	SAP-5
DESCRIPTION	Getting Max Staking Rewards Possible While Bypassing the Lockup Periods
SEVERITY	Medium ⓘ

STATUS

Fixed

ID

SAP-6

DESCRIPTION

Signature Malleability in Unused
`recoverSigner` Function

SEVERITY

• Medium ⓘ

STATUS

Acknowledged

ID

SAP-7

DESCRIPTION

Missing Access Control on Token
Release Function

SEVERITY

• Medium ⓘ

STATUS

Fixed

ID

SAP-8

DESCRIPTION

`calculateMultiplier` Will Always
Return `maxMultiplier`

SEVERITY

• Medium ⓘ

STATUS

Acknowledged

ID

SAP-9

DESCRIPTION

Vesting Can Be Updated in Any Way
Desired by the Multisig

SEVERITY

• Low ⓘ

STATUS

Fixed

ID

SAP-10

DESCRIPTION

Missing `_disableInitializers()`
Call in the Constructor

SEVERITY

• Low ⓘ

STATUS

Fixed

ID

SAP-11

DESCRIPTION

Vesting Could Transfer Tokens to the
Original Gnosis Safe

SEVERITY

• Low ⓘ

STATUS

Acknowledged

ID

SAP-12

DESCRIPTION

The Received Allocations Might Not Be
the Expected Ones.

SEVERITY

• Low ⓘ

STATUS

Fixed

ID

SAP-13

DESCRIPTION	Test Suite Cannot Be Executed
SEVERITY	Low ⓘ
STATUS	Acknowledged

ID	SAP-14
DESCRIPTION	Inequitable Multiplier Distribution in <code>calculateMultiplier()</code> Function
SEVERITY	Low ⓘ
STATUS	Acknowledged

ID	SAP-15
DESCRIPTION	Event Is Not Emitted when Transaction Reverts
SEVERITY	Informational ⓘ
STATUS	Fixed

ID	SAP-16
DESCRIPTION	Redundant Inheritance of <code>OwnableUpgradeable</code>
SEVERITY	Informational ⓘ
STATUS	Fixed

ID	SAP-17
DESCRIPTION	Boolean State of VestingSchedule.revoked Never Modified
SEVERITY	• Informational ⓘ
STATUS	Fixed

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.



Disclaimer

Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following

1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.

2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
 1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
 2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

Files Included

Repo: [https://github.com/Sapien-io/sapien-contracts\(0b386c7\)](https://github.com/Sapien-io/sapien-contracts(0b386c7)) Files: contracts/*

Files Excluded

All other files.

Key Actors And Their Capabilities

The Admin

The admin of the protocol is allowed to set the vesting schedules and to release the staking rewards.

Users

Users are able to stake and unstake in the protocol.

Findings

SAP-1

Missing Balance Deduction in Unstaking Functions Allows Contract Drainage and Lock Period Bypass

• **High** ⓘ **Fixed**



Update

Marked as "Fixed" by the client.

Addressed in: `4a45a4a` .

✓ Update

Marked as "Fixed" by the client.

Addressed in: `f647099b7988fa4d01fac6a9f3481d25e748527a` .

The client provided the following explanation:

The vulnerability in the unstaking functions was fixed by implementing proper balance deduction and state management.

✓ Update

Marked as "Fixed" by the client.

Addressed in: `f647099b7988fa4d01fac6a9f3481d25e748527a` .

The client provided the following explanation:

The vulnerability in the unstaking functions was fixed by implementing proper balance deduction and state management.

File(s) affected: `SapienStaking.sol`

Description: The `initiateUnstake()` and `unstake()` functions in the staking contract contain a critical vulnerability that allows users to drain the entire contract's token balance. When unstaking, the functions fail to deduct the staked amount from the user's balance in the `StakingInfo` struct. This means a user can:

1. Stake tokens once to create a valid staking position
2. Call `initiateUnstake()` to start the cooldown period
3. After the cooldown period, call `unstake()` multiple times with the same `orderId` to repeatedly withdraw tokens
4. Each withdrawal succeeds since the staking position remains active and the balance is never decremented

As a result, a malicious user could drain all tokens from the contract by repeatedly calling `unstake()` with a single valid staking position, far exceeding their initial stake.

This also tricks the reward mechanism off-chain, because the user `isActive` state variable is never updated, which signals that the user stake is still active, as well as having the original amount that was staked.

In addition to the above, the lock duration is not checked during unstaking. Meaning that an attacker can game the reward system by staking for the maximum duration, and then unstake immediately.

Exploit Scenario:

1. Alice stakes 100 tokens and receives a valid `orderId` and signature
2. Alice initiates unstaking by calling `initiateUnstake()`
3. After the cooldown period completes, Alice calls `unstake()` to withdraw 100 tokens
4. Since her staking position remains active and unchanged, Alice can call `unstake()` again
5. Alice can repeat this process until the contract's entire token balance is drained

Recommendation: Modify the `unstake()` function to properly update the user's staking position. Also, add checks for the lockdown duration.

SAP-2

Signature Replay Attack Possible Between Stake, Unstake and Reward Functions Enabling Unauthorized Token Claims

• High ⓘ Fixed

✓ Update

Marked as "Fixed" by the client.
Addressed in: `fa79159` .

✓ Update

Marked as "Fixed" by the client.
Addressed in: `4ba050c5db3fe00d6c39cad790a6d95834e2a624` .
The client provided the following explanation:

The vulnerability was fixed by implementing EIP-712 typed structured data signing in both contracts.

✓ Update

Marked as "Fixed" by the client.
Addressed in: `4ba050c5db3fe00d6c39cad790a6d95834e2a624` .
The client provided the following explanation:

The vulnerability was fixed by implementing EIP-712 typed structured data signing in both contracts.

File(s) affected: `SapienStaking.sol` , `Rewards.sol`

Description: A critical signature replay vulnerability exists in the `SapienRewards` and

`SapienStaking` contracts. The `verifyOrder()` function in both contracts uses the same signature format and validation logic:

```
bytes32 messageHash = keccak256(abi.encodePacked(userWallet,
rewardAmount, orderId));
```

This allows signatures to be reused across different contract functions. Specifically:

1. A signature generated for staking tokens can be replayed to claim rewards in the `SapienRewards` contract
2. The same signature can also be replayed multiple times for staking and unstaking within `SapienStaking`

This means an attacker who obtains a valid signature meant for staking could:

- Use it to claim unauthorized rewards from `SapienRewards`
- Drain tokens through repeated unstaking operations (see [SAP-1](#))
- Accidentally overwrite existing staking positions, causing themselves to lose their staked tokens

The vulnerability allows:

- Unauthorized claiming of rewards using stake signatures
- Multiple drains of the same stake amount
- Overwriting of legitimate user stakes

Exploit Scenario:

Attacker claims to want to stake 1000 tokens and requests a signed order:

- userWallet: attacker address
 - amount: 1000 tokens
 - orderId: "ORDER_1000_1"
 - signature: signed by sapienAddress for staking
1. Other than claimed the attacker use the signature in `SapienRewards: claimReward(1000, "ORDER_1000_1", signature)`
 2. The transaction succeeds because:
 - msg.sender matches the userWallet in the signature
 - The amount matches
 - The orderId hasn't been used in `SapienRewards`
 - The signature verification passes since it contains the same parameters
 3. Result: Attacker claims 1000 tokens in rewards using a signature that was only intended for staking

Recommendation: 1. Implement EIP-712 typed structured data signing-

2. Track used orders to prevent replay.

This fix should be implemented in both contracts to prevent cross-contract and cross-function replay attacks.

SAP-3

Bloom Filter Implementation Can Lead to False Positives Preventing Valid Reward Claims

• Medium ⓘ

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: 73264fb .

✓ Update

Marked as "Fixed" by the client.

Addressed in: 34c39e3637c09d1d5d850bae1897d1200c23cfef .

The client provided the following explanation:

```
The Bloom filter implementation for tracking redeemed orders was replaced with a simple double mapping structure (mapping(address => mapping(bytes32 => bool)) private redeemedOrders).
```

✓ Update

Marked as "Fixed" by the client.

Addressed in: 34c39e3637c09d1d5d850bae1897d1200c23cfef .

The client provided the following explanation:

```
The Bloom filter implementation for tracking redeemed orders was replaced with a simple double mapping structure (mapping(address => mapping(bytes32 => bool)) private redeemedOrders).
```

File(s) affected: Rewards.sol

Description: The Rewards contract uses a Bloom filter to track redeemed order IDs, but this implementation can lead to false positives that prevent users from claiming legitimate rewards. The issue arises from the limited size of the filter (256 bits) combined with multiple hash insertions per order.

Each order sets 3 bits in the 256-bit filter. With users expected to claim up to 1000 rewards, this results in up to 3000 bits being set in a 256-bit space, leading to a high probability of collisions.

Impact

- As users claim more rewards, the Bloom filter becomes increasingly saturated
- This increases the probability of hash collisions causing false positives
- Valid orders may be incorrectly identified as already redeemed

- Users will be unable to claim legitimate rewards once collisions occur

Exploit Scenario:

1. After claiming ~300 rewards:
 - Each claim triggered 3 hash operations
 - Many bits were hit multiple times
 - The majority of the 256 bits in Alice's filter are now set to 1
2. Alice attempts to claim a new legitimate reward: `claimReward(amount, "REWARD_1000", signature)`
3. The transaction reverts because:
 - The 3 hash positions for this orderId happen to map to bits that were already set by previous claims
 - `isOrderRedeemed()` returns `true` (false positive)
 - The legitimate claim is blocked

Recommendation: Replace the complex Bloom filter implementation with a simple mapping to track redeemed orders. Even high-volume contracts like USDC rely on simple mappings for state tracking such as user balances due to their reliability and reasonable gas costs. An example could be the following:

```
mapping(address => mapping(bytes32 => bool)) private redeemedOrders;

function isOrderRedeemed(address user, string calldata orderId)
    internal view returns (bool) {
    return redeemedOrders[user][keccak256(abi.encodePacked(orderId))];
}

function addOrderToRedeemed(address user, string calldata orderId)
    internal {
    redeemedOrders[user][keccak256(abi.encodePacked(orderId))] = true;
}
```

SAP-4

Emergency Pause Functionality Not Implemented

• **Medium** ⓘ **Fixed**



Update

Marked as "Fixed" by the client.
Addressed in: `fcf703d`.

✓ **Update**

Marked as "Fixed" by the client.

Addressed in: `f99f1d2ce37b2710534182bb2166b648dbb14d5c` .

The client provided the following explanation:

The vulnerability was fixed by implementing the missing `pause()` and `unpause()` functions in the Rewards contract.

✓ **Update**

Marked as "Fixed" by the client.

Addressed in: `f99f1d2ce37b2710534182bb2166b648dbb14d5c` .

The client provided the following explanation:

The vulnerability was fixed by implementing the missing `pause()` and `unpause()` functions in the Rewards contract.

File(s) affected: `Rewards.sol`

Description: The `Rewards` contract inherits from `PausableUpgradeable` and includes `pause` checks in critical functions, but does not implement the actual `pause()` and `unpause()` functions. This means the contract cannot be paused in case of an emergency, despite functions being marked with `whenNotPaused` modifier.

Recommendation: Implement pause functionality by adding the `pause()` and `unpause()` functions.

SAP-5

Getting Max Staking Rewards Possible While Bypassing the Lockup Periods

• **Medium** ⓘ **Fixed**

✓ **Update**

Marked as "Fixed" by the client.

Addressed in: `b175349` .

✓ **Update**

Marked as "Fixed" by the client.

Addressed in: `228ae219c5478f375bed56376ffba8538ea2f09e` .

The client provided the following explanation:

The vulnerability was fixed by adding lock period validation checks across unstaking functions.

✓ Update

Marked as "Fixed" by the client.

Addressed in: 228ae219c5478f375bed56376ffba8538ea2f09e .

The client provided the following explanation:

The vulnerability was fixed by adding lock period validation checks across unstaking functions.

File(s) affected: SapienStaking.sol

Description: The staking contract determines the multiplier a user gets depending on the staking amount and the lockup period. Where the lockup period is the most important factor since it determines the max multiplier. However, any of the unstake-related functions check if the lockup period has actually elapsed. There are two consequences:

1. A user can stake the needed amount for the max multiplier for 12 months to get the maximum rewards but only lock up the funds for two days after calling `initiateUnstake()` immediately after staking.
2. A user can mistakenly call `instantUnstake()` after the lockup period has ended which would cost an unnecessary penalty.

Recommendation: We recommend to check that the lockup period has ended before allowing the user to initiate the unstaking process. Additionally, make sure the `instantUnstake()` can only be called before the lockup period has ended to avoid unnecessary penalties.

SAP-6

Signature Malleability in Unused `recoverSigner` Function

• Medium ⓘ

Acknowledged

✓ Update

Marked as "Fixed" by the client.

Addressed in: b175349 .

The client provided the following explanation:

fixed as part of SAP-2

✓ Update

Marked as "Fixed" by the client.

Addressed in: `6c76d2a49d97a826d9eaec37673213672add9633` .

The client provided the following explanation:

The vulnerability was fixed by removing the unused `recoverSigner()` function that relied on the vulnerable `ecrecover` precompile. Instead, the contract now uses OpenZeppelin's ECDSA library for signature verification.

✓ Update

Marked as "Fixed" by the client.

Addressed in: `6c76d2a49d97a826d9eaec37673213672add9633` .

The client provided the following explanation:

The vulnerability was fixed by removing the unused `recoverSigner()` function that relied on the vulnerable `ecrecover` precompile. Instead, the contract now uses OpenZeppelin's ECDSA library for signature verification.

File(s) affected: `Rewards.sol`

Description: The `recoverSigner()` function in the `Rewards` contract uses the `ecrecover` precompile which is vulnerable to signature malleability. This vulnerability allows multiple valid signatures to be generated for the same message, as `ecrecover` accepts signatures with both high and low values of the `s` parameter.

While this would typically be a high-severity issue that could allow signature replay attacks, the severity is significantly reduced because the function is currently unused in the contract. But since the contract is upgradeable consider fixing this issue to avoid impacting newer versions.

Recommendation: Avoid using this function and instead ensure the OpenZeppelin `ECDSA` library is used, which makes the necessary checks to ensure signature malleability is not possible.

SAP-7

Missing Access Control on Token Release Function

• Medium ⓘ Fixed

✓ **Update**

Marked as "Fixed" by the client.

Addressed in: `ab75f15` .

✓ **Update**

Marked as "Fixed" by the client.

Addressed in: `2814cd4533d30b7bcf50721f34eb348edeb3b493` .

The client provided the following explanation:

The vulnerability was fixed by implementing proper access control in the `SapTestToken` contract.

✓ **Update**

Marked as "Fixed" by the client.

Addressed in: `2814cd4533d30b7bcf50721f34eb348edeb3b493` .

The client provided the following explanation:

The vulnerability was fixed by implementing proper access control in the `SapTestToken` contract.

File(s) affected: `SapienRewards.sol`, `SapTestToken.sol`

Description: The `releaseTokens()` function in the `SapTestToken` contract lacks access control, allowing any address to call it. The intention is that it is only callable via the `SapienRewards.releaseTokenRewards()` function by the owner of that contract. However, a user is able to bypass this and instead directly call `releaseTokens()` on the `SapTestToken` contract.

Recommendation: Consider adding a check within `SapTestToken.releaseTokens()` to ensure that the caller is the `SapienRewards` contract address.

SAP-8 `calculateMultiplier` **Will Always Return** `maxMultiplier`

• **Medium** ⓘ

Acknowledged

✓ **Update**

Marked as "Fixed" by the client.

Addressed in: `9bc4417` .

✓ Update

Marked as "Fixed" by the client.

Addressed in: 2dbe4303d2566707230b741c18ec2ca7b779b9a1 .

The client provided the following explanation:

The calculateMultiplier() function (lines 454–490 in SapienStaking.sol) implements a tiered multiplier system that works correctly for amounts both above and below BASE_STAKE.

✓ Update

Marked as "Fixed" by the client.

Addressed in: 2dbe4303d2566707230b741c18ec2ca7b779b9a1 .

The client provided the following explanation:

The calculateMultiplier() function (lines 454–490 in SapienStaking.sol) implements a tiered multiplier system that works correctly for amounts both above and below BASE_STAKE.

File(s) affected: SapienStaking.sol

Description: Currently, the function calculateMultiplier() will always return the maxMultiplier if the following condition is true:

```
if (amount >= BASE_STAKE) {  
    return maxMultiplier;  
}
```

However, this condition will always be true, since during staking, it is required that the amount staked be greater than the BASE_STAKE :

```
require(amount >= BASE_STAKE, "Amount must be greater than base stake");
```

Recommendation: Consider reworking this function to prevent the maximum multiplier from being exploited.

SAP-9

Vesting Can Be Updated in Any Way Desired by the Multisig

• Low ⓘ

Fixed

✓ **Update**

Marked as "Fixed" by the client.
Addressed in: 18f31d5 .

✓ **Update**

Marked as "Fixed" by the client.
Addressed in: 6693aad88cbf3c96c83d7dffcab138c4e770ad8a .
The client provided the following explanation:

The vulnerability was fixed by adding validation checks in the `updateVestingSchedule()` function.

✓ **Update**

Marked as "Fixed" by the client.
Addressed in: 6693aad88cbf3c96c83d7dffcab138c4e770ad8a .
The client provided the following explanation:

The vulnerability was fixed by adding validation checks in the `updateVestingSchedule()` function.

File(s) affected: SapTestToken.sol

Description: The `updateVestingSchedule()` function allows the Gnosis Safe to modify any parameter of a vesting schedule without restrictions. This includes:

- Setting `amount` to 0 even after tokens have been released or reducing it below what has already been released creates an inconsistent state.
- Setting the `duration` lower than the `cliff` period, which breaks the vesting logic. Users would have to wait for the cliff to be able to claim everything at once instead of periodically.
- Modifying start time to a future date even after vesting has begun also creates an inconsistent state.
- Changing the safe address mid-vesting could break integration with other contracts expecting specific safe addresses.

Recommendation: Evaluate if it should be possible to update the vesting amount, start, duration, cliff and destination after the vesting has already begun, if it should be possible to update the amount to zero at any time or reduce the amount to a value lower than what has already been released. If not, we recommend adding the proper checks. Also, evaluate if the schedules should be updated at all.

SAP-10

Missing `_disableInitializers()` Call in the Constructor

• Low ⓘ

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `5754DB9` .

✓ Update

Marked as "Fixed" by the client.

Addressed in: `55c3880b64b39f94f2d85cc640940c78e3554645` .

The client provided the following explanation:

The `_disableInitializers()` function is now being called in the constructors which now exist.

✓ Update

Marked as "Fixed" by the client.

Addressed in: `55c3880b64b39f94f2d85cc640940c78e3554645` .

The client provided the following explanation:

The `_disableInitializers()` function is now being called in the constructors which now exist.

File(s) affected: `SapTestToken.sol` , `Rewards.sol` , `SapienStaking.sol`

Description: To prevent initialization of the implementation contract, lock it down by calling `_disableInitializers()` in the constructor. Read more in the OpenZeppelin documentation for the [initializable contract](#).

Recommendation: Add `constructors` with a call to `_disableInitializers()` to all contracts.

SAP-11

Vesting Could Transfer Tokens to the Original Gnosis Safe

• Low ⓘ

Acknowledged

Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

```
The entities will have separate safe addresses on mainnet
```

Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

```
This suggestion didn't involve immediate action but it has been noted for the deployment process.
```

Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

```
This suggestion didn't involve immediate action but it has been noted for the deployment process.
```

Description: The `releaseTokens()` function transfers the tokens from the `gnosisSafe` (who originally got all the token supply) to the `schedule.safe` address which originally was set as the same `gnosisSafe`. This could lead to the contract and the original receiving team thinking that a vesting was successfully released when the original owner of the token still has them.

Recommendation: Avoid minting the tokens beforehand and only mint when it is needed.

SAP-12

The Received Allocations Might Not Be the Expected Ones.

• Low ⓘ

Fixed

Update

Marked as "Fixed" by the client.

Addressed in: 3161944 .

✓ **Update**

Marked as "Fixed" by the client.

Addressed in: d12a86a0dc8ff02043d00b6f60b0a1823c08d4f3 .

The client provided the following explanation:

The contract is now checking to make sure that the total supply is not less than the sum of the expected allocations.

✓ **Update**

Marked as "Fixed" by the client.

Addressed in: d12a86a0dc8ff02043d00b6f60b0a1823c08d4f3 .

The client provided the following explanation:

The contract is now checking to make sure that the total supply is not less than the sum of the expected allocations.

File(s) affected: SapTestToken.sol

Description: The `initialize()` function mints the amount of `_totalSupply` . However, there is no guarantee that it is the sum of all the expected allocations (1B tokens). If `_totalSupply` is less than the sum of all allocations, the vesting schedules will be created with amounts that can never be fully claimed. If `_totalSupply` is greater than 1B, the excess tokens will be minted to the Gnosis Safe but won't be part of any vesting schedule.

Recommendation: Make sure that `_totalSupply` is equal to the sum of all the expected allocations. In case, it should be greater, modify the contract to reflect that and update the documentation.

SAP-13 Test Suite Cannot Be Executed

• Low ⓘ Acknowledged

i **Update**

Marked as "Acknowledged" by the client.

The client provided the following explanation:

fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `a66bb9ef5d8fafc1cefb56c58fba3d695075a261`.

The client provided the following explanation:

The test suite has been updated and is passing, it is implemented in multiple commits, so please review from latest.

✓ Update

Marked as "Fixed" by the client.

Addressed in: `a66bb9ef5d8fafc1cefb56c58fba3d695075a261`.

The client provided the following explanation:

The test suite has been updated and is passing, it is implemented in multiple commits, so please review from latest.

Description: According to the README, the test suite can be executed by running `npm run test`. However, the tests are actually written with the syntax expected by Truffle, not Hardhat. As a result the above mentioned command throws an error. Unfortunately, using Truffle does not work either since running `truffle test` also throws an error.

Recommendation: A comprehensive test suite is crucial for smart contract development as these contracts are immutable once deployed and often handle significant financial value. Tests help catch vulnerabilities and logic errors before deployment, preventing potential losses, while also serving as living documentation of the system's intended behavior. Moreover, as contracts evolve, tests ensure that new changes don't break existing functionality, which is especially important for upgradeable contracts where new implementations must maintain expected behavior. Additionally, make sure to have at least 90% code coverage.

SAP-14

Inequitable Multiplier Distribution in `calculateMultiplier()` Function

• Low ⓘ Acknowledged

i Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

On mainnet, the requirement for a minimum BASE_STAKE will be removed, and the multiplier will be adjusted accordingly to prevent gaming the system by staking very little over a long period to achieve the maximum multiplier.

The client provided the following explanation:

The client provided the following explanation:

- Informational ⓘ Fixed

✓ **Update**

Marked as "Fixed" by the client.
Addressed in: `dd79d7f` .

✓ **Update**

Marked as "Fixed" by the client.
Addressed in: `3f9a00380d668b7879b62d0fa86ed1c43cdb9736` .
The client provided the following explanation:

```
Premature event emission removed and appropriate event emission  
now happening.
```

✓ **Update**

Marked as "Fixed" by the client.
Addressed in: `3f9a00380d668b7879b62d0fa86ed1c43cdb9736` .
The client provided the following explanation:

```
Premature event emission removed and appropriate event emission  
now happening.
```

File(s) affected: `Rewards.sol`

Description: The `claimReward()` function emits a `WithdrawalProcessed` event before potentially reverting the transaction. This event emission is redundant as all state changes including events are rolled back when a transaction reverts in the EVM. In the case where `success` is `false`, the `WithdrawalProcessed` event will never be visible on-chain.

Recommendation: Remove the premature event emission and only emit events after confirming the operation's success.

SAP-16 Redundant Inheritance of `OwnableUpgradeable`

• **Informational** ⓘ

Fixed

✓ **Update**

Marked as "Fixed" by the client.
Addressed in: `1b2dc63` .

✓ **Update**

Marked as "Fixed" by the client.

Addressed in: `6e709c1891cad3620aa7f64e2d29b60ba159aa6f` .

The client provided the following explanation:

`Inheritance of OwnableUpgradeable removed.`



Update

Marked as "Fixed" by the client.

Addressed in: `6e709c1891cad3620aa7f64e2d29b60ba159aa6f` .

The client provided the following explanation:

`Inheritance of OwnableUpgradeable removed.`

File(s) affected: `SapTestToken.sol`

Description: The `SapTestToken` contract inherits from `OwnableUpgradeable` but makes no use of its functionality.

Recommendation: Consider removing the inheritance from `OwnableUpgradeable` if there are no plans to use its functionality in the codebase.

SAP-17

Boolean State of `VestingSchedule.revoked` Never Modified

• Informational ⓘ

Fixed



Update

Marked as "Fixed" by the client.

Addressed in: `cb094f8` .



Update

Marked as "Fixed" by the client.

Addressed in: `8a0a205de922c611702750e4c9a57e9503f409cc` .

The client provided the following explanation:

`VestingSchedule.revoked` removed from the token contract.



Update

Marked as "Fixed" by the client.

Addressed in: `8a0a205de922c611702750e4c9a57e9503f409cc` .

The client provided the following explanation:

```
VestingSchedule.revoked removed from the token contract.
```

File(s) affected: `SapTestToken.sol`

Description: The `revoked` field in the `VestingSchedule` struct is present but can never be set to true as there are no functions that modify this state variable. This creates dead code and may indicate missing functionality for revoking vesting schedules.

Recommendation: Consider removing this unused code if there are no plans to use its functionality in the codebase.

Auditor Suggestions

S1

Contract Must Be Pre-Funded for Users to Claim Rewards

Acknowledged

Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

```
token releases will be manual at this point
```

Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

```
This input has been noted and the server will actively manage the  
funding of rewards.
```

Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

This input has been noted and the server will actively manage the funding of rewards.

File(s) affected: Rewards.sol

Description: The SapienRewards contract requires manual funding to fulfill reward claims, as evidenced by the balance check in the hasTokenBalance modifier. If the contract is not adequately pre-funded users with valid claim signatures will be unable to claim their rewards

Recommendation: Whenever the server signs a reward order, it should also fund the contract accordingly to ensure users are always able to claim rewards.

S2 Missing Input Validation

Fixed

✓ Update

Marked as "Fixed" by the client.
Addressed in: d130668 .

✓ Update

Marked as "Fixed" by the client.
Addressed in: 663e5755ba34046f0488119daa2956929978342d .
The client provided the following explanation:

Recommended validations added to contract.

✓ Update

Marked as "Fixed" by the client.
Addressed in: 663e5755ba34046f0488119daa2956929978342d .
The client provided the following explanation:

Recommended validations added to contract.

File(s) affected: SapTestToken.sol , SapienStaking.sol

Description: SapTestToken

1. `updateVestingSchedule()`
 - `allocationType` : Validate vesting schedule for allocation type exists.
 - `cliff` : Validate cliff is smaller than `duration` .
 - `start` : Is greater than `block.timestamp` .

SapienStaking

1. `initialize`
 - `_sapienToken` : Check for zero address.
 - `_sapienAddress` : Check for zero address.
1. `unstake`
 - `amount` : Is greater than zero.

Recommendation: Add the recommended validations.

S3 Unlocked Pragma

Fixed



Update

Marked as "Fixed" by the client.

Addressed in: `7942ee4` .



Update

Marked as "Fixed" by the client.

Addressed in: `0ff558595d0b28b5e27f06fb1dfdc3cdc4883ba6` .

The client provided the following explanation:

`Carats removed, effectively locking pragma version.`



Update

Marked as "Fixed" by the client.

Addressed in: `0ff558595d0b28b5e27f06fb1dfdc3cdc4883ba6` .

The client provided the following explanation:

`Carats removed, effectively locking pragma version.`

Related Issue(s): [SWC-103](#)

Description: Every Solidity file specifies in the header a version number of the format `pragma solidity (^)0.8.*` . The caret (`^`) before the version number implies an unlocked pragma, meaning that the compiler will use the specified version *and above*, hence the term "unlocked".

Recommendation: For consistency and to prevent unexpected behavior in the future, we recommend to remove the caret to lock the file onto a specific Solidity version.

S4 General Improvements

Acknowledged

✓ Update

Marked as "Fixed" by the client.
Addressed in: `f910c37` .

i Update

Marked as "Mitigated" by the client.
Addressed in: `e73efe7bdef6c7ebea94d9db58c32f8b99410ecd` .
The client provided the following explanation:

DECIMALS value added, but immutability not an option,
functionally equivalent require statement added. We received
approval from Mostafa for implement this way.

i Update

Marked as "Mitigated" by the client.
Addressed in: `e73efe7bdef6c7ebea94d9db58c32f8b99410ecd` .
The client provided the following explanation:

DECIMALS value added, but immutability not an option,
functionally equivalent require statement added. We received
approval from Mostafa for implement this way.

File(s) affected: `SapienStaking.sol` , `SapTestToken.sol`

Description: - Replace magic number 18 in the `SapTestToken` contract by variable `DECIMALS` .

- `SapienStaking.sapienToken` can be immutable.
- `SapienStaking.sapenAddress` can be immutable.

Recommendation: Consider addressing all the suggestions mentioned above.

S5 Remove Unused Struct Member `VestingSchedule.revoked`

Fixed



Update

Marked as "Fixed" by the client.

Addressed in: f910c37 .



Update

Marked as "Fixed" by the client.

Addressed in: 8a0a205de922c611702750e4c9a57e9503f409cc .

The client provided the following explanation:

Removed as a part of a previous suggestion.



Update

Marked as "Fixed" by the client.

Addressed in: 8a0a205de922c611702750e4c9a57e9503f409cc .

The client provided the following explanation:

Removed as a part of a previous suggestion.

File(s) affected: SapTestToken.sol

Description: The revoked parameter is not used anywhere and can be removed.

S6 Remove Redundant lockUpPeriod Check

Fixed



Update

Marked as "Fixed" by the client.

Addressed in: 1d5bbfb .



Update

Marked as "Fixed" by the client.

Addressed in: 37d43f43726247eb4feb4ea9f0043b4ea8584ad2 .

The client provided the following explanation:

Redundant check removed.

✓ Update

Marked as "Fixed" by the client.

Addressed in: 37d43f43726247eb4feb4ea9f0043b4ea8584ad2 .

The client provided the following explanation:

Redundant check removed.

File(s) affected: SapienStaking.sol

Description: The `SapienStaking.stake()` function checks the `lockUpPeriod` twice. Once in the beginning of the function and once in `getMaxMultiplier()` function.

Recommendation: Remove the check in the beginning of the `SapienStaking.stake()` function and only check the validity in the `getMaxMultiplier()` function.

S7 Check Return Values of Token Transfers

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: 008ef2e .

✓ Update

Marked as "Fixed" by the client.

Addressed in: 8318230c4211302a3ffa42573c5206a6e95b5e38 .

The client provided the following explanation:

The vulnerability was fixed by adding explicit return value checks for all token transfer operations in both contracts.

✓ Update

Marked as "Fixed" by the client.

Addressed in: 8318230c4211302a3ffa42573c5206a6e95b5e38 .

The client provided the following explanation:

The vulnerability was fixed by adding explicit return value checks for all token transfer operations in both contracts.

File(s) affected: `SapienStaking.sol`, `Rewards.sol`

Description: The ERC20 standard requires to check the return value of `transfer()` and `transferFrom()` function calls. The OpenZeppelin implementation of the Sapien token used here automatically reverts when the transfer was not successful.

Recommendation: To stick to the standard and to prevent any errors in the future check the boolean return value of the function calls.

S8 Use Most Similar Contract Interface

Fixed



Update

Marked as "Fixed" by the client.

Addressed in: `b86c814`.



Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

In this case, I would argue that the current implementation is actually better than using the specific token contract type for several reasons:

Loose Coupling: Using an interface keeps the contract loosely coupled. We can use any token that implements both the IERC20 standard and the `releaseTokens` function.

Flexibility: If we need to upgrade or change the token contract in the future, we won't need to modify this contract as long as the new token implements the same interface.

Testing: It's easier to mock and test with interfaces.

Gas Efficiency: Interfaces are typically more gas efficient as they only include the function signatures you need.

This audit suggestion would be more relevant if we were using just IERC20 while needing specific functions from `SapTestToken`. However, we've already created a custom interface (`IRewardToken`) that includes exactly what we need.

Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

In this case, I would argue that the current implementation is actually better than using the specific token contract type for several reasons:

Loose Coupling: Using an interface keeps the contract loosely coupled. We can use any token that implements both the IERC20 standard and the releaseTokens function.

Flexibility: If we need to upgrade or change the token contract in the future, we won't need to modify this contract as long as the new token implements the same interface.

Testing: It's easier to mock and test with interfaces.

Gas Efficiency: Interfaces are typically more gas efficient as they only include the function signatures you need.

This audit suggestion would be more relevant if we were using just IERC20 while needing specific functions from SapTestToken. However, we've already created a custom interface (IRewardToken) that includes exactly what we need.

File(s) affected: Rewards.sol

Description: The SapienStaking contract stores the sapienToken as a IERC20 variable. This is not wrong but the IERC20 is a very general interface. Because the sapienToken will always be the SapTestToken use the SapTestToken instead of the IERC20 interface.

S9

Function Break Checks-Effects-Interactions Pattern

Fixed

Update

Marked as "Fixed" by the client.

Addressed in: c8ba11d .

Update

Marked as "Fixed" by the client.

Addressed in: 18075b494b63a10dd2a3130125a11feca21ef16d .

The client provided the following explanation:

Fixed function formatting to make sure that it is following the CEI pattern.

✓ Update

Marked as "Fixed" by the client.

Addressed in: `18075b494b63a10dd2a3130125a11feca21ef16d` .

The client provided the following explanation:

Fixed function formatting to make sure that it is following the CEI pattern.

File(s) affected: `SapienStaking.sol` , `Rewards.sol`

Description: The Checks-Effects-Interactions (CEI) pattern is recommended to use to minimize the attack surface to reentrancy attacks. Although most functions are using the `nonReentrant` modifier to prevent reentrancy attacks, it is still recommended to follow the CEI function structure.

The following functions do not follow CEI:

- `SapienStaking.stake()`
- `Rewards.claimReward()`

S10 Remove Unused Functions

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `c8ba11d` .

✓ Update

Marked as "Fixed" by the client.

Addressed in: `18f19811aa5dfb8005479c5019791303d4fd57dd` .

The client provided the following explanation:

Removed unused functions. One of the functions I believe was removed as a natural extension of a prior commit.

✓ Update

Marked as "Fixed" by the client.

Addressed in: 18f19811aa5dfb8005479c5019791303d4fd57dd .

The client provided the following explanation:

```
Removed unused functions. One of the functions I believe was
removed as a natural extension of a prior commit.
```

File(s) affected: Rewards.sol

Description: Remove the unused functions in the SapientRewards contract:

- getMessageHash()
- recoverSigner()
- splitSignature()

S11 Remove hasTokenBalance() Modifier

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: 021683b .

The client provided the following explanation:

```
and 949c651
```

✓ Update

Marked as "Fixed" by the client.

Addressed in: c09872f8f7a0a776b596fc336948d629ffd4cdfa .

The client provided the following explanation:

```
Removed the modifier and just performed the operation explicitly
in the only instance of use.
```

✓ Update

Marked as "Fixed" by the client.

Addressed in: c09872f8f7a0a776b596fc336948d629ffd4cdfa .

The client provided the following explanation:

```
Removed the modifier and just performed the operation explicitly
```

in the only instance of use.

File(s) affected: Rewards.sol

Description: The modifier is only used once and could a check the `SapientRewards.claimReward()` function.

S12

Contract Layout Does Not Follow Solidity Style Guide

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: d1e95de .

The client provided the following explanation:

and e0d366d

✓ Update

Marked as "Fixed" by the client.

Addressed in: 72e08796480b69ab9214659f75b505aec04b10e2 .

The client provided the following explanation:

I made an effort to rearrange what I thought was in violation of the style guide. Please advise on any further improvements.

✓ Update

Marked as "Fixed" by the client.

Addressed in: 72e08796480b69ab9214659f75b505aec04b10e2 .

The client provided the following explanation:

I made an effort to rearrange what I thought was in violation of the style guide. Please advise on any further improvements.

File(s) affected: Rewards.sol

Description: The `SapientRewards` contract's organization deviates from the standard Solidity

style guide, with variables between function definitions. This makes the code harder to read and maintain.

Recommendation: Reorganize contract elements according to the [style guide order](#).

S13

Use `SapTestToken` Interface Instead of Custom `IRewardToken`

Fixed



Update

Marked as "Fixed" by the client.
Addressed in: `d1e95de` .



Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

Please see `rebuttal` on S-8



Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

Please see `rebuttal` on S-8

File(s) affected: `Rewards.sol`

Description: The contract defines a custom `IRewardToken` interface but will only interact with `SapTestToken` . Consider importing and using the `SapTestToken` contract interface directly to ensure type safety and avoid potential interface mismatches.

S14 Pass Reward Token in Initializer

Acknowledged



Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

we need to set the vesting wallet for rewards in the token contract so the rewards contract it needs to be deployed first

i Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

I see it as functionally reasonable to maintain the ability of the contract to switch an award token. Please advise if I am missing something.

i Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

I see it as functionally reasonable to maintain the ability of the contract to switch an award token. Please advise if I am missing something.

File(s) affected: Rewards.sol

Description: Since the SapTestToken will be the only token in use I can be passed in the SapienRewards.initializer() function and the SapienRewards.setRewardToken() can be removed.

S15 Remove SapienRewards.releaseRewardTokens() Function

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: d1e95de .

✓ Update

Marked as "Fixed" by the client.

Addressed in: 68635ab0036e26289612a83fbd49897b88fe2823 .

The client provided the following explanation:

This function was removed.

✓ Update

Marked as "Fixed" by the client.

Addressed in: 68635ab0036e26289612a83fbd49897b88fe2823 .

The client provided the following explanation:

This function was removed.

File(s) affected: Rewards.sol

Description: There is no apparent use-case to call the `rewardToken.releaseTokens()` function from the `SapienRewards` contract since it could be called directly on the `SapTestToken` contract.

S16

Critical Role Transfer Not Following Two-Step Pattern

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: 18f31d5 .

✓ Update

Marked as "Fixed" by the client.

Addressed in: 653f42faf8539315bbc59455120bd37c2765dfc8 .

The client provided the following explanation:

Two step approval pattern implemented.

✓ Update

Marked as "Fixed" by the client.

Addressed in: 653f42faf8539315bbc59455120bd37c2765dfc8 .

The client provided the following explanation:

Two step approval pattern implemented.

Description: The owner of the contracts can call `transferOwnership()` to transfer the ownership to a new address. If an uncontrollable address is accidentally provided as the new owner address then the contract will no longer have an active owner, and functions with the `onlyOwner` modifier can no longer be executed.

Recommendation: Consider using OpenZeppelin's `Ownable2Step` contract to adopt a two-step ownership pattern in which the new owner must accept their position before the transfer is complete.

S17 Missing Input Validation

Fixed

✓ Update

Marked as "Fixed" by the client.
Addressed in: `d8c89d0` .

✓ Update

Marked as "Fixed" by the client.
Addressed in: `bc39d1d25bde41fa1235ecd3978e72571e680404` .
The client provided the following explanation:

Input validation added for the three instances noted.

✓ Update

Marked as "Fixed" by the client.
Addressed in: `bc39d1d25bde41fa1235ecd3978e72571e680404` .
The client provided the following explanation:

Input validation added for the three instances noted.

File(s) affected: `SapienStaking.sol` , `Rewards.sol`

Related Issue(s): [SWC-123](#)

Description: It is important to validate inputs, even if they only come from trusted addresses, to

avoid human error. The following is a non-exhaustive list of missing checks:

1. The `SapienStaking.initialize()` function should check the address of both inputs against `address(0)`.
2. The `Rewards.initialize()` function should check that `_authorizedSigner` is not `address(0)`.
3. The `SapTestToken.updateVestingSchedule()` does not perform any input validation. This could allow to have a duration shorter than the cliff, amount equal to zero or a safe address equal to `address(0)`.

Recommendation: We recommend adding the relevant checks.

S18 General Recommendations

Mitigated

✓ Update

Marked as "Fixed" by the client.
Addressed in: `fd1db5c`.

✓ Update

Marked as "Fixed" by the client.
Addressed in: `3810f5fa0ea850517eb91db51663090e4790861a`.
The client provided the following explanation:

All changes implemented except the previously stated inability to make `sapienToken` and `sapienAddress` immutable.

✓ Update

Marked as "Fixed" by the client.
Addressed in: `3810f5fa0ea850517eb91db51663090e4790861a`.
The client provided the following explanation:

All changes implemented except the previously stated inability to make `sapienToken` and `sapienAddress` immutable.

Description: This provides general suggestions and best practices to improve the overall security, efficiency, and maintainability of the contract. These recommendations address potential optimizations, enhancements, and preventive measures that are not tied to specific vulnerabilities but contribute to the robustness of the system.

1. Strings are relatively expensive in Solidity, especially when used for storage or passed as

function arguments, because they are dynamically sized and require additional gas to handle their variable-length encoding and decoding. In contrast, bytes32 is much more efficient because it is a fixed-size data type, making it cheaper to store and manipulate.

2. Consider emitting events at the end of the function or after important executions. The `SapTestToken.initialize()` function emits the event `InitializedEvent()` at the very beginning.
3. The codebase is using truffle as development tool specifically for tests. Unfortunately, truffle was shutdown over a year ago and a migration to a more modern and user-friendly tool is recommended. Consider fully migrating to hardhat since the codebase is already using it.
4. Remove dead or unused code for better code readability and comprehension. Additionally, in the case of solidity, it might also increase the bytecode size increasing deployment costs.

The following is a non-exhaustive list of unused code:

1. The events `SignatureVerified` and `RecoveredSigner` in the `Rewards` contract are not used.
2. The functions `getMessageHash()`, `recoverSigner()` and `splitSignature()` in the `Rewards` contract are not used.
5. Certain state variables that are set only once could be declared as immutable and be set at construction time which would be a bit more gas efficient. The following is a non-exhaustive list of variables that be declared immutable and be set once in the constructor:
 1. The variables `sapienToken` and `sapienAddress` in the `SapienStaking` contract.
 2. The variables `vestingStartTimestamp` and `gnosisSafe` in the `StapTestToken` contract.
 3. The variable `authorizedSigner` in the `Rewards` contract.
6. The `SapTestToken` contract sets the `_gnosisSafe` as the owner, in which case, it is unnecessary to have another modifier `onlySafe` when the `onlyOwner` modifier already exists.

Recommendation: Consider addressing all the suggestions mentioned above.

S19 `allocationType` Could Be Changed to an `enum`

Fixed



Update

Marked as "Fixed" by the client.

Addressed in: `dc65124` .



Update

Marked as "Fixed" by the client.

Addressed in: `e26c15668893107bc7e80f74eba707bf81b76bda` .

The client provided the following explanation:

Allocation type changed to enum.

✓ Update

Marked as "Fixed" by the client.

Addressed in: e26c15668893107bc7e80f74eba707bf81b76bda .

The client provided the following explanation:

Allocation type changed to enum.

File(s) affected: SapTestToken.sol

Description: The SapTestToken allocates rewards to different groups based on an allocationType which is a string type. This means that the protocol could later call updateVestingSchedule passing an allocationType that was not present in the original _createHardcodedVestingSchedules(). Instead, using an enum of the recipients of the vested tokens would be a better way to "hardcode" the contract's vesting details. It would also reduce the possibility of error and reduce the amount of trust required from users.

Recommendation: Consider creating an allocation type enum such as the following:

```
enum AllocationType {  
    INVESTORS,  
    TEAM,  
    REWARDS,  
    AIRDROP,  
    COMMUNITY_TREASURY,  
    STAKING_INCENTIVES,  
    LIQUIDITY_INCENTIVES  
}
```

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or

is a risk that the client has indicated is low impact in view of the client's business circumstances.

- **Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Appendix

File Signatures

The following are the SHA-256 hashes of the reviewed files. A file with a different SHA-256 hash has been modified, intentionally or otherwise, after the security review. You are cautioned that a different SHA-256 hash could be (but is not necessarily) an indication of a changed condition or potential vulnerability that was not within the scope of the review.

Files

- 3e0...996 ./contracts/Rewards.sol
- fcc...405 ./contracts/SapientStaking.sol
- da0...376 ./contracts/SapTestToken.sol

Tests

- 29a...7a0 ./test/Rewards.test.js
- e95...900 ./test/SapientStaking.test.js
- 7a4...507 ./test/SapTestToken.test.js
- 5d3...af2 ./test/unit1.js

Test Suite Results

The code testing suite uses Truffle, which is long deprecated. It is recommended to migrate to hardhat.

```
0 passing (4s)
1 failing
```

Code Coverage

The code testing suite uses Truffle, which is long deprecated. It is recommended to migrate to hardhat.

Changelog

- 2025-01-23 - Initial report
- 2025-02-05 - Fix review

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 500 audits and secured over \$200 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Polygon, Avalanche, Curve, Solana, Compound, Lido, MakerDAO, Arbitrum, OpenSea and the World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Near, Flow, Avalanche, Solana, Cardano, Binance Smart Chain, Hedera Hashgraph, Tezos
- DeFi: Curve, Compound, Maker, Lido, Polygon, Arbitrum, SushiSwap
- NFT: OpenSea, Parallel, Dapper Labs, Decentraland, Sandbox, Axie Infinity, Illuvium, NBA Top Shot, Zora
- Academic institutions: National University of Singapore, MIT

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any

report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party

software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.



© 2026 – Quantstamp, Inc.

Sapien